



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course: Fundamentals of Data Science

Course Code: DSA0404

Slot: A

Faculty: KRISHNARAJ M

Submitted By

Y. Sravan Kumar (192472289)

CO1 AT3- Data Exploratory based Practical Assessment

1. House Average Price Using NumPy

Code:

```
house_average1.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/house_average1.py (3.12.3)
File Edit Format Run Options Window Help

import numpy as np

house_data = np.loadtxt(
    "House_data1.csv",
    delimiter=";",
    skiprows=1,
    dtype=str
)

bedrooms = house_data[:, 1].astype(int)
sale_price = house_data[:, 3].astype(float)

houses_more_than_4 = sale_price[bedrooms > 4]

average_price = np.mean(houses_more_than_4)

print("Average Sale Price:", round(average_price, 2))
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/house_average1.py
Average Sale Price: 15500000.0
>>>
```

2. Logistic Regression (Spam Detection)

Code:

```
spam_detection.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/spam_detection.py (3.12.3)
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Load the dataset
df = pd.read_csv("Email_data.csv")

# Features (Input)
X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]

# Target (Output)
y = df["Spam"]

# Create and train the model
model = LogisticRegression()
model.fit(X, y)

# User Input
links = int(input("Enter Number of Links: "))
caps = int(input("Enter Number of Capitalized Words: "))
length = int(input("Enter Email Length: "))

# Create a DataFrame for the new email
new_email = pd.DataFrame({
    "Num_Links": [links],
    "Num_Caps_Words": [caps],
    "Email_Length": [length]
})

# Predict
prediction = model.predict(new_email)

# Display Result
if prediction[0] == 1:
    print("Prediction: Spam Email")
else:
    print("Prediction: Not Spam Email")
|
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/spam_detection.py
Enter Number of Links: 5
Enter Number of Capitalized Words: 12
Enter Email Length: 260
Prediction: Spam Email
>>>
```

3. Logistic Regression (Same Dataset)

Code:

spam_detection.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/spam_detection.py (3.12.3)

```
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Load the dataset
df = pd.read_csv("Email_data.csv")

# Features (Input)
X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]

# Target (Output)
y = df["Spam"]

# Create and train the model
model = LogisticRegression()
model.fit(X, y)

# User Input
links = int(input("Enter Number of Links: "))
caps = int(input("Enter Number of Capitalized Words: "))
length = int(input("Enter Email Length: "))

# Create a DataFrame for the new email
new_email = pd.DataFrame({
    "Num_Links": [links],
    "Num_Caps_Words": [caps],
    "Email_Length": [length]
})

# Predict
prediction = model.predict(new_email)

# Display Result
if prediction[0] == 1:
    print("Prediction: Spam Email")
else:
    print("Prediction: Not Spam Email")
|
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/spam_detection.py
Enter Number of Links: 1
Enter Number of Capitalized Words: 2
Enter Email Length: 100
Prediction: Not Spam Email
>>>|
```

4. Linear Regression (Car Resale Price)

Code:

```

car_resale_predection.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/car_resale_predection.py (3.12.3)
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load dataset
df = pd.read_csv("Car_data.csv")

# Features (Input)
X = df[["Age", "Kms_Driven", "Engine_CC"]]

# Target (Output)
y = df["Resale_Price"]

# Train the model
model = LinearRegression()
model.fit(X, y)

# User Input
age = int(input("Enter Age of Car: "))
kms = int(input("Enter Kilometers Driven: "))
engine = int(input("Enter Engine CC: "))

# Create DataFrame for prediction
new_car = pd.DataFrame({
    "Age": [age],
    "Kms_Driven": [kms],
    "Engine_CC": [engine]
})

# Predict resale price
predicted_price = model.predict(new_car)

print("Estimated Resale Price: ₹", round(predicted_price[0], 2))

```

Output:

```

IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/car_resale_predection.py
Enter Age of Car: 4
Enter Kilometers Driven: 50000
Enter Engine CC: 1400
Estimated Resale Price: ₹ 495263.16
>>>

```

5. CART Decision Tree (Loan Approval)

Code:

loan_approval.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/loan_approval.py (3.12.3)

```
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

# Load dataset
df = pd.read_csv("Loan_data.csv")

# Features (Input)
X = df[["Income", "Credit_Score", "Existing_Loans"]]

# Target (Output)
y = df["Approved"]

# Train the model
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X, y)

# User Input
income = int(input("Enter Income: "))
credit = int(input("Enter Credit Score: "))
loans = int(input("Enter Existing Loans: "))

# Create a DataFrame for the new applicant
new_applicant = pd.DataFrame({
    "Income": [income],
    "Credit_Score": [credit],
    "Existing_Loans": [loans]
})

# Predict
prediction = model.predict(new_applicant)

# Display Result
if prediction[0] == 1:
    print("\nLoan Approved")
else:
    print("\nLoan Rejected")

# Display Decision Rules
print("\nDecision Rules:\n")
print(export_text(model, feature_names=list(X.columns)))
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/FDS lab/loan_approval.py
Enter Income: 50000
Enter Credit Score: 730
Enter Existing Loans: 1

Loan Approved

Decision Rules:

|--- Credit_Score <= 680.00
| |--- class: 0
|--- Credit_Score > 680.00
| |--- class: 1
>>>
```